

```

#Nidhi Mangtani 23070521098
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.optimizers import Adam

# Load MNIST Dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize images
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Fixed Hyperparameters
learning_rate = 0.01
batch_size = 32
epochs = 5
activation = "relu"

# Build Model
model = Sequential()
model.add(Flatten(input_shape=(28, 28)))

# Hidden Layers (64 neurons each)
model.add(Dense(64, activation=activation))
model.add(Dense(64, activation=activation))
model.add(Dense(64, activation=activation))
model.add(Dense(64, activation=activation))

# Output Layer
model.add(Dense(10, activation="softmax"))

# Compile Model
model.compile(
    optimizer=Adam(learning_rate=learning_rate),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# Display Model Summary
model.summary()

# Train Model
history = model.fit(
    x_train,
    y_train,
    epochs=epochs,
    batch_size=batch_size,
    validation_split=0.2,
    verbose=1
)

# Evaluate Model
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)

# Print Results
print("\n===== Model Performance =====")
print(f"Training Accuracy : {history.history['accuracy'][-1]:.4f}")
print(f"Validation Accuracy : {history.history['val_accuracy'][-1]:.4f}")
print(f"Test Accuracy : {test_accuracy:.4f}")
print(f"Test Loss : {test_loss:.4f}")

```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>

11490434/11490434 — 0s 0us/step

/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input\_shape`/
super().__init__(**kwargs)

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 64)	50,240
dense_1 (Dense)	(None, 64)	4,160
dense_2 (Dense)	(None, 64)	4,160
dense_3 (Dense)	(None, 64)	4,160
dense_4 (Dense)	(None, 10)	650

Total params: 63,370 (247.54 KB)

Trainable params: 63,370 (247.54 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/5

1500/1500 — 7s 4ms/step - accuracy: 0.8996 - loss: 0.3501 - val_accuracy: 0.9280 - val_loss: 0.2947

Epoch 2/5

1500/1500 — 9s 3ms/step - accuracy: 0.9400 - loss: 0.2296 - val_accuracy: 0.9389 - val_loss: 0.2305

Epoch 3/5

1500/1500 — 7s 5ms/step - accuracy: 0.9484 - loss: 0.2051 - val_accuracy: 0.9523 - val_loss: 0.2039

Epoch 4/5

1500/1500 — 5s 3ms/step - accuracy: 0.9532 - loss: 0.1884 - val_accuracy: 0.9559 - val_loss: 0.1774

Epoch 5/5

1500/1500 — 8s 5ms/step - accuracy: 0.9567 - loss: 0.1765 - val_accuracy: 0.9573 - val_loss: 0.1825

===== Model Performance =====

Training Accuracy : 0.9567

Validation Accuracy : 0.9573

Test Accuracy : 0.9583

Test Loss : 0.1771

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.optimizers import Adam
```

```
# Load MNIST Dataset
```

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

```
# Normalize images
```

```
x_train = x_train.astype("float32") / 255.0
```

```
x_test = x_test.astype("float32") / 255.0
```

```
# Fixed Hyperparameters
```

```
learning_rate = 0.001
```

```
batch_size = 64
```

```
epochs = 10
```

```
activation = "tanh"
```

```
# Build Model
```

```
model = Sequential()
```

```
model.add(Flatten(input_shape=(28, 28)))
```

```
# Hidden Layers (64 neurons each)
```

```
model.add(Dense(64, activation=activation))
```

```
model.add(Dense(64, activation=activation))
```

```
model.add(Dense(64, activation=activation))
```

```
model.add(Dense(64, activation=activation))
```

```
# Output Layer
```

```
model.add(Dense(10, activation="softmax"))
```

```
# Compile Model
```

```
model.compile(
```

```
    optimizer=Adam(learning_rate=learning_rate),
```

```
    loss="sparse_categorical_crossentropy",
```

```
    metrics=["accuracy"]
```

```
)
```

```
# Display Model Summary
```

```
model.summary()
```

```
# Train Model
```

```
history = model.fit(
```

```
    x_train,
```

```

y_train,
epochs=epochs,
batch_size=batch_size,
validation_split=0.2,
verbose=1
)

# Evaluate Model
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)

# Print Results
print("\n===== Model Performance =====")
print(f"Training Accuracy : {history.history['accuracy'][-1]:.4f}")
print(f"Validation Accuracy : {history.history['val_accuracy'][-1]:.4f}")
print(f"Test Accuracy : {test_accuracy:.4f}")
print(f"Test Loss : {test_loss:.4f}")

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape` /
super().__init__(**kwargs)

```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
flatten_1 (Flatten)	(None, 784)	0
dense_5 (Dense)	(None, 64)	50,240
dense_6 (Dense)	(None, 64)	4,160
dense_7 (Dense)	(None, 64)	4,160
dense_8 (Dense)	(None, 64)	4,160
dense_9 (Dense)	(None, 10)	650

Total params: 63,370 (247.54 KB)

Trainable params: 63,370 (247.54 KB)

Non-trainable params: 0 (0.00 B)

```

Epoch 1/10
750/750 ————— 4s 4ms/step - accuracy: 0.9036 - loss: 0.3358 - val_accuracy: 0.9433 - val_loss: 0.1934
Epoch 2/10
750/750 ————— 3s 4ms/step - accuracy: 0.9536 - loss: 0.1540 - val_accuracy: 0.9593 - val_loss: 0.1345
Epoch 3/10
750/750 ————— 4s 5ms/step - accuracy: 0.9664 - loss: 0.1112 - val_accuracy: 0.9637 - val_loss: 0.1229
Epoch 4/10
750/750 ————— 3s 4ms/step - accuracy: 0.9737 - loss: 0.0862 - val_accuracy: 0.9679 - val_loss: 0.1078
Epoch 5/10
750/750 ————— 3s 4ms/step - accuracy: 0.9783 - loss: 0.0689 - val_accuracy: 0.9674 - val_loss: 0.1114
Epoch 6/10
750/750 ————— 3s 4ms/step - accuracy: 0.9827 - loss: 0.0557 - val_accuracy: 0.9675 - val_loss: 0.1071
Epoch 7/10
750/750 ————— 3s 4ms/step - accuracy: 0.9844 - loss: 0.0487 - val_accuracy: 0.9597 - val_loss: 0.1421
Epoch 8/10
750/750 ————— 4s 5ms/step - accuracy: 0.9868 - loss: 0.0416 - val_accuracy: 0.9686 - val_loss: 0.1079
Epoch 9/10
750/750 ————— 3s 4ms/step - accuracy: 0.9895 - loss: 0.0338 - val_accuracy: 0.9668 - val_loss: 0.1223
Epoch 10/10
750/750 ————— 4s 6ms/step - accuracy: 0.9900 - loss: 0.0304 - val_accuracy: 0.9698 - val_loss: 0.1079

```

===== Model Performance =====

Training Accuracy : 0.9900

Validation Accuracy : 0.9698

Test Accuracy : 0.9709

Test Loss : 0.1026

```

import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.optimizers import Adam

# Load MNIST Dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize images
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Fixed Hyperparameters
learning_rate = 0.5
batch_size = 256
epochs = 3
activation = "linear"

# Build Model
model = Sequential()

```

```

model.add(Flatten(input_shape=(28, 28)))

# Hidden Layers (64 neurons each)
model.add(Dense(64, activation=activation))
model.add(Dense(64, activation=activation))
model.add(Dense(64, activation=activation))
model.add(Dense(64, activation=activation))

# Output Layer
model.add(Dense(10, activation="softmax"))

# Compile Model
model.compile(
    optimizer=Adam(learning_rate=learning_rate),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# Display Model Summary
model.summary()

# Train Model
history = model.fit(
    x_train,
    y_train,
    epochs=epochs,
    batch_size=batch_size,
    validation_split=0.2,
    verbose=1
)

# Evaluate Model
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)

# Print Results
print("\n===== Model Performance =====")
print(f"Training Accuracy : {history.history['accuracy'][-1]:.4f}")
print(f"Validation Accuracy : {history.history['val_accuracy'][-1]:.4f}")
print(f"Test Accuracy : {test_accuracy:.4f}")
print(f"Test Loss : {test_loss:.4f}")

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input\_shape`
super().__init__(**kwargs)

Model: "sequential_2"

Layer (type)	Output Shape	Param #
flatten_2 (Flatten)	(None, 784)	0
dense_10 (Dense)	(None, 64)	50,240
dense_11 (Dense)	(None, 64)	4,160
dense_12 (Dense)	(None, 64)	4,160
dense_13 (Dense)	(None, 64)	4,160
dense_14 (Dense)	(None, 10)	650

Total params: 63,370 (247.54 KB)

Trainable params: 63,370 (247.54 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/3

188/188 ————— 4s 12ms/step - accuracy: 0.6479 - loss: 1500725.8750 - val_accuracy: 0.8397 - val_loss: 7633.45

Epoch 2/3

188/188 ————— 1s 6ms/step - accuracy: 0.8477 - loss: 5091.3145 - val_accuracy: 0.8139 - val_loss: 3368.0239

Epoch 3/3

188/188 ————— 1s 6ms/step - accuracy: 0.8545 - loss: 2636.0603 - val_accuracy: 0.8475 - val_loss: 2433.7463

===== Model Performance =====

Training Accuracy : 0.8545

Validation Accuracy : 0.8475

Test Accuracy : 0.8488

Test Loss : 2649.6482

Model Performance Comparison (All Scenarios)

Metric	Original Hyperparameters	First Updated Hyperparameters	'Bad Results' Hyperparameters
Hyperparameters			
Learning Rate	0.01	0.001	0.5
Batch Size	32	64	256
Epochs	5	10	3
Activation	'relu'	'tanh'	'linear'

Metric	Original Hyperparameters	First Updated Hyperparameters	'Bad Results' Hyperparameters
Performance			
Training Accuracy	0.9567	0.9999	0.8654
Validation Accuracy	0.9573	0.9780	0.8643
Test Accuracy	0.9583	0.9709	0.8488
Test Loss	0.1771	0.1026	2649.6482